Problem 1/20

SESSIONIZATION

META

Session Gap

> Identify new user sessions when inactivity exceeds 30 minutes.

SAMPLE INPUT

user_id	event_time
U-1045	2023-10-01 10:00
U-1045	2023-10-01 10:15
U-1045	2023-10-01 11:10

EXPECTED OUTPUT

user_id	event_time	session_id
U-1045	2023-10-01 10:00	1
U-1045	2023-10-01 10:15	1
U-1045	2023-10-01 11:10	2

SOLUTION

SQL SERVER

```sql
SELECT user_id, event_time,
SUM(is_new) OVER (PARTITION BY user_id ORDER BY event_time) AS session_id
FROM (
  SELECT user_id, event_time,
  CASE
    WHEN DATEDIFF(minute, LAG(event_time) OVER
      (PARTITION BY user_id ORDER BY event_time), event_time) > 30
    THEN 1
    ELSE 0
  END AS is_new
  FROM events
) t;
```

Next: Growth Trend →

Anandnarayanan S

The 20-Pattern SQL Cheatsheet

SAVE

REPOST

Problem 2/20

LAG / LEAD

ADOBE

Growth Trend

Find customers whose spend increases month over month.

SAMPLE INPUT

customer_id	month	amount
C-992	2023-01	50.00
C-992	2023-02	120.00
C-114	2023-01	80.00
C-114	2023-02	70.00

EXPECTED OUTPUT

customer_id
C-992

SOLUTION

SQL SERVER

```sql
SELECT DISTINCT customer_id
FROM (
  SELECT customer_id, month, amount,
  LAG(amount) OVER (PARTITION BY customer_id ORDER BY month) AS prev_amt
  FROM spend
) t
WHERE amount > prev_amt;
```

Next: Back-to-Back →

Anandnarayanan S

The 20-Pattern SQL Cheatsheet

SAVE

REPOST

Problem 3/20

WINDOW FILTER

ZOMATO

Back-to-Back

Find users who placed orders on consecutive days.

SAMPLE INPUT

customer_id	order_date
C-105	2023-11-03
C-105	2023-11-04
C-202	2023-11-10

EXPECTED OUTPUT

customer_id
C-105

SOLUTION

SQL SERVER

```sql
SELECT DISTINCT customer_id
FROM (
  SELECT customer_id, order_date,
  LAG(order_date) OVER
    (PARTITION BY customer_id ORDER BY order_date) AS prev_date
  FROM orders
) t
WHERE DATEDIFF(day, prev_date, order_date) = 1;
```

Next: Churned Users →

Anandnarayanan S

The 20-Pattern SQL Cheatsheet

SAVE

REPOST

Problem 4/20

NOT IN

SPOTIFY

Churned Users

Find customers active last month but not this month.

SAMPLE INPUT

customer_id	activity_date
U-801	2024-02-15
U-801	2024-03-05
U-802	2024-02-20

EXPECTED OUTPUT

customer_id
U-802

SOLUTION

STANDARD SQL

```sql
SELECT DISTINCT a.customer_id
FROM activity a
WHERE MONTH(a.activity_date) = MONTH(CURDATE() - INTERVAL 1 MONTH)
AND YEAR(a.activity_date) = YEAR(CURDATE() - INTERVAL 1 MONTH)
AND a.customer_id NOT IN (
  SELECT customer_id
  FROM activity
  WHERE MONTH(activity_date) = MONTH(CURDATE())
  AND YEAR(activity_date) = YEAR(CURDATE())
);
```

Next: Loyal Users →

Anandnarayanan S

The 20-Pattern SQL Cheatsheet

SAVE

REPOST

Problem 5/20

HAVING / COUNT

NETFLIX

Loyal Users

Identify customers who placed orders in every available month.

SAMPLE INPUT

customer_id	order_date
C-44	2023-01-10
C-44	2023-02-15
C-55	2023-01-20

EXPECTED OUTPUT

customer_id
C-44

SOLUTION

MYSQL

```mysql
SELECT customer_id
FROM orders
GROUP BY customer_id
HAVING COUNT(DISTINCT MONTH(order_date)) =
  (SELECT COUNT(DISTINCT MONTH(order_date)) FROM orders);
```

Next: Missing IDs →

Anandnarayanan S

The 20-Pattern SQL Cheatsheet

SAVE

REPOST

Problem 6/20

LEFT JOIN

PAYTM

Missing IDs

Find missing order IDs in a sequential numeric list.

SAMPLE INPUT

id
1001
1002
1004
1005

EXPECTED OUTPUT

missing_id
1003

SOLUTION

MYSQL

```mysql
SELECT o1.id + 1 AS missing_id
FROM orders o1
LEFT JOIN orders o2 ON o1.id + 1 = o2.id
WHERE o2.id IS NULL
AND o1.id < (SELECT MAX(id) FROM orders);
```

Next: 3-Day Streak →

Anandnarayanan S

The 20-Pattern SQL Cheatsheet

SAVE

REPOST

Problem 7/20

GAPS & ISLANDS

META

3-Day Streak

Find users active for at least 3 consecutive days.

SAMPLE INPUT

user_id	login_date
U-77	2023-10-01
U-77	2023-10-02
U-77	2023-10-03
U-88	2023-10-01

EXPECTED OUTPUT

user_id
U-77

SOLUTION

SQL SERVER

```sql
SELECT user_id
FROM (
  SELECT user_id, login_date,
  DATEADD(day, -ROW_NUMBER() OVER
    (PARTITION BY user_id ORDER BY login_date), login_date) AS grp
  FROM logins
) t
GROUP BY user_id, grp
HAVING COUNT(*) ≥ 3;
```

Next: Share of Total →

Anandnarayanan S

The 20-Pattern SQL Cheatsheet

SAVE

REPOST

Problem 8/20

WINDOW RATIO

ADOBE

Share of Total

Calculate category-wise percentage distribution of revenue.

SAMPLE INPUT

category	revenue
Phones	50000
Laptops	150000

EXPECTED OUTPUT

category	pct
Phones	25.0
Laptops	75.0

SOLUTION

STANDARD SQL

```sql
SELECT
  category,
  SUM(revenue) * 100.0 / SUM(SUM(revenue)) OVER () AS pct
FROM sales
GROUP BY category;
```

Next: Delivery SLA →

Anandnarayanan S

The 20-Pattern SQL Cheatsheet

SAVE

REPOST

Problem 9/20

CASE WHEN

SWIGGY

Delivery SLA

Categorize orders based on delivery duration (Fast <= 2 Days).

SAMPLE INPUT

order_id	order_date	delivery_date
ORD-1	2023-12-01	2023-12-02
ORD-2	2023-12-01	2023-12-05

EXPECTED OUTPUT

order_id	delivery_type
ORD-1	Fast
ORD-2	Slow

SOLUTION

SQL SERVER

```sql
SELECT
  order_id,
  CASE
    WHEN DATEDIFF(day, order_date, delivery_date) ≤ 2
THEN 'Fast'
    ELSE 'Slow'
  END AS delivery_type
FROM orders;
```

Next: Org Chart →

Anandnarayanan S

The 20-Pattern SQL Cheatsheet

SAVE

REPOST

Problem 10/20

SELF JOIN

INFOSYS

Org Chart

Fetch employee, manager, and senior manager names.

SAMPLE INPUT

name	manager_id	id
Bob Smith	102	101
Alice Lee	103	102
Sarah CEO	NULL	103

EXPECTED OUTPUT

emp_name	mgr_name	sr_mgr_name
Bob Smith	Alice Lee	Sarah CEO

SOLUTION

SQL SERVER

```sql
SELECT
  e.name AS emp_name,
  m.name AS mgr_name,
  sm.name AS sr_mgr_name
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.id
LEFT JOIN employees sm ON m.manager_id = sm.id;
```

Next: Failed Payments →

Anandnarayanan S

The 20-Pattern SQL Cheatsheet

SAVE

REPOST

Problem 11/20

NOT EXISTS

RAZORPAY

Failed Payments

Find orders that have no matching payments.

SAMPLE INPUT

id	total_amount
ORD-99	100.00
ORD-100	50.00

EXPECTED OUTPUT

id
ORD-100

SOLUTION

MYSQL

```mysql
SELECT o.id
FROM orders o
WHERE NOT EXISTS (
  SELECT 1 FROM payments p
  WHERE p.order_id = o.id
);
```

Next: Row to Column →

Anandnarayanan S

The 20-Pattern SQL Cheatsheet

SAVE

REPOST

Problem 12/20

MANUAL PIVOT

DELOITTE

Row to Column

Convert row-based monthly sales into aggregated columns.

SAMPLE INPUT

month	sales
Jan	15000
Feb	25000
Mar	10000

EXPECTED OUTPUT

Jan	Feb	Mar
15000	25000	10000

SOLUTION

MYSQL

```mysql
SELECT
  SUM(CASE WHEN month='Jan' THEN sales END) AS Jan,
  SUM(CASE WHEN month='Feb' THEN sales END) AS Feb,
  SUM(CASE WHEN month='Mar' THEN sales END) AS Mar
FROM sales;
```

Next: Double Booking →

Anandnarayanan S

The 20-Pattern SQL Cheatsheet

SAVE

REPOST

Problem 13/20

THETA JOIN

AIRBNB

Double Booking

Identify overlapping booking intervals for properties.

SAMPLE INPUT

id	start_date	end_date
B-10	2023-06-01	2023-06-05
B-11	2023-06-04	2023-06-10

EXPECTED OUTPUT

id	start_date	end_date
B-10	2023-06-01	2023-06-05
B-11	2023-06-04	2023-06-10

SOLUTION

SQL SERVER

```sql
SELECT a.*
FROM bookings a
JOIN bookings b
ON a.id <> b.id
AND a.start_date < b.end_date
AND a.end_date > b.start_date;
```

Next: True Middle →

Anandnarayanan S

The 20-Pattern SQL Cheatsheet

SAVE

REPOST

Problem 14/20

MATH WINDOW

ORACLE

True Middle

Calculate the true mathematical median of employee salaries.

SAMPLE INPUT

salary
60000
70000
75000
80000
120000

EXPECTED OUTPUT

median
75000

SOLUTION

STANDARD SQL

```sql
SELECT AVG(salary) AS median
FROM (
  SELECT salary,
  ROW_NUMBER() OVER (ORDER BY salary) AS rn,
  COUNT(*) OVER () AS cnt
  FROM employees
) t
WHERE rn IN ((cnt+1)/2, (cnt+2)/2);
```

Next: 2-Month Active →

Anandnarayanan S

The 20-Pattern SQL Cheatsheet

SAVE

REPOST

Problem 15/20

GROUP FILTER

SPOTIFY

2-Month Active

Find users active in both Month 1 and Month 2.

SAMPLE INPUT

user_id	month
U-500	2024-01
U-500	2024-02
U-600	2024-01

EXPECTED OUTPUT

user_id
U-500

SOLUTION

MYSQL

```mysql
SELECT DISTINCT user_id
FROM activity
WHERE month IN ('2024-01', '2024-02')
GROUP BY user_id
HAVING COUNT(DISTINCT month) = 2;
```

Next: Cross-Dept Avg →

Anandnarayanan S

The 20-Pattern SQL Cheatsheet

SAVE

REPOST

Problem 16/20

ADV WINDOW

AMAZON

Cross-Dept Avg

Find employees earning more than the company avg (excluding their own dept).

SAMPLE INPUT

emp_name	dept	salary
John	Sales	120000
Jane	IT	80000
Mike	IT	60000

EXPECTED OUTPUT

emp_name
John

SOLUTION

STANDARD SQL

```sql
SELECT emp_name FROM (
  SELECT emp_name, salary,
  (SUM(salary) OVER() - SUM(salary) OVER(PARTITION BY dept)) /
  NULLIF(COUNT(*) OVER() - COUNT(*) OVER(PARTITION BY dept), 0) AS
out_avg
  FROM employees
) t
WHERE salary > out_avg;
```

Next: Wallet Balance →

Anandnarayanan S

The 20-Pattern SQL Cheatsheet

SAVE

REPOST

Problem 17/20

RUNNING TOTAL

PAYTM

Wallet Balance

Calculate final user balance from credits (CR) and debits (DB).

SAMPLE INPUT

user_id	type	amount
U-123	CR	150.00
U-123	DB	45.00
U-123	CR	20.00

EXPECTED OUTPUT

user_id	balance
U-123	125.00

SOLUTION

STANDARD SQL

```sql
SELECT
  user_id,
  SUM(CASE WHEN type = 'CR' THEN amount ELSE -amount END)
AS balance
FROM transactions
GROUP BY user_id;
```

Next: Top 2 Salaries →

Anandnarayanan S

The 20-Pattern SQL Cheatsheet

SAVE

REPOST

Problem 18/20

DENSE_RANK

UBER

Top 2 Salaries

Find the top 2 highest paid employees per department.

SAMPLE INPUT

dept	salary
Eng	150000
Eng	140000
Eng	120000

EXPECTED OUTPUT

dept	salary
Eng	150000
Eng	140000

SOLUTION

STANDARD SQL

```sql
SELECT dept, salary FROM (
  SELECT dept, salary,
  DENSE_RANK() OVER(PARTITION BY dept ORDER BY salary
DESC) AS rnk
  FROM employees
) t
WHERE rnk ≤ 2;
```

Next: 7-Day Avg →

Anandnarayanan S

The 20-Pattern SQL Cheatsheet

SAVE

REPOST

Problem 19/20

ROLLING SUM

STRIPE

7-Day Avg

Calculate a 7-day rolling average of daily revenue.

SAMPLE INPUT

date	revenue
2023-11-01	1000
2023-11-02	2000
2023-11-03	3000

EXPECTED OUTPUT

date	rolling_avg
2023-11-01	1000.0
2023-11-02	1500.0
2023-11-03	2000.0

SOLUTION

POSTGRES

```sql
SELECT
  date,
  AVG(revenue) OVER(
    ORDER BY date
    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
  ) AS rolling_avg
FROM daily_sales;
```

Next: MISSION ACCOMPLISHED →

Anandnarayanan S

The 20-Pattern SQL Cheatsheet

SAVE

REPOST

MISSION ACCOMPLISHED

You are Interview Ready!

20 PATTERNS MASTERED 100%

- [x] SESSIONIZATION
- [x] LAG / LEAD
- [x] WINDOW FILTER
- [x] NOT IN
- [x] HAVING / COUNT
- [x] LEFT JOIN
- [x] GAPS & ISLANDS
- [x] WINDOW RATIO
- [x] CASE WHEN
- [x] SELF JOIN
- [x] NOT EXISTS
- [x] MANUAL PIVOT
- [x] THETA JOIN
- [x] MATH WINDOW
- [x] GROUP FILTER
- [x] ADV WINDOW
- [x] RUNNING TOTAL
- [x] DENSE_RANK
- [x] ROLLING SUM

FOLLOW FOR MORE ♥

Found this helpful?

Help your network learn SQL the easy way!

REPOST